

30 Node.js Interview Questions

with concise answers and code examples

Core Basics

Q1. What is Node.js?

Node.js is an open-source, cross-platform JavaScript runtime built on Chrome's V8 engine. It lets you run JavaScript outside the browser, mainly for servers, CLIs and tooling, using an event-driven, non-blocking I/O model.

Q2. Is Node.js single-threaded?

Your JavaScript runs on a single main thread, but Node is not purely single-threaded. libuv maintains a thread pool (default size 4) for file system, DNS, crypto and zlib work, and the OS handles network I/O asynchronously. You can also use worker threads and child processes.

Q3. What is the difference between Node.js and browser JavaScript?

Browsers provide the DOM, window and Web APIs. Node provides fs, http, path, process and other system-level modules, has no DOM, and uses module systems (CommonJS/ESM) natively.

Q4. What is npm and what is package.json?

npm is Node's package manager and registry. package.json describes the project: name, version, scripts, dependencies, devDependencies and entry point. package-lock.json pins exact dependency versions for reproducible installs.

Q5. dependencies vs devDependencies?

dependencies are needed at runtime in production. devDependencies are only for development (testing, linting, bundlers). npm install --omit=dev skips devDependencies.

Q6. What is the difference between require and import?

require is CommonJS: synchronous, dynamic, and can be called anywhere. import is ES Modules: statically analyzable, asynchronous loading, supports top-level await. Use "type": "module" in package.json or the .mjs extension to enable ESM.

Q7. What is the difference between module.exports and exports?

exports is just a reference to module.exports. Adding properties to exports works, but reassigning exports (exports = ...) breaks the link, so to export a single function or class you must assign module.exports.

```
module.exports = function hello() {}; // works
exports = function hello() {};      // does NOT work
```

Q8. What are global objects in Node.js?

Objects available everywhere without import, such as process, Buffer, console, setTimeout, setImmediate, global and globalThis. In CommonJS, __dirname and __filename are module-scoped, not truly global.

Async and the Event Loop

Q9. What is the event loop?

The event loop is the mechanism that lets Node perform non-blocking I/O. It continuously checks queues and runs callbacks when the call stack is empty, offloading heavy operations to the OS or the libuv thread pool.

Q10. Name the phases of the event loop.

In order: timers (setTimeout/setInterval), pending callbacks, idle/prepare, poll (I/O events), check (setImmediate), and close callbacks. Between each phase, process.nextTick and promise microtasks are drained.

Q11. process.nextTick vs setImmediate vs setTimeout(fn, 0)?

process.nextTick runs right after the current operation, before the event loop continues (highest priority). Promise callbacks run in the microtask queue just after. setImmediate runs in the check phase. setTimeout(fn, 0) runs in the timers phase. Outside an I/O callback, the order of setTimeout 0 and setImmediate is not guaranteed; inside an I/O callback, setImmediate always fires first.

Q12. Callbacks, Promises and async/await: how do they differ?

Callbacks are the original pattern but lead to nested 'callback hell'. Promises represent a future value and support chaining and .catch. async/await is syntax on top of promises that makes asynchronous code read sequentially and allows try/catch error handling.

```
async function load() {
  try {
    const data = await fs.promises.readFile('a.txt', 'utf8');
    return data;
  } catch (err) {
    console.error(err);
  }
}
```

Q13. Promise.all vs allSettled vs race vs any?

all: resolves with all results, rejects on the first rejection. allSettled: waits for all and reports each outcome. race: settles with the first promise to settle. any: resolves with the first fulfilled promise, rejects only if all reject.

Q14. What is blocking vs non-blocking code?

Blocking code (e.g. fs.readFileSync, heavy loops) stops the event loop until it finishes, delaying all other requests. Non-blocking code starts the operation and continues, handling the result in a callback or promise later.

Q15. How do you handle CPU-intensive tasks in Node.js?

Move them off the main thread: use worker_threads, child_process or cluster, break work into chunks with setImmediate, or delegate to a separate service or queue.

Q16. What is libuv?

libuv is the C library that gives Node its event loop, thread pool and cross-platform asynchronous I/O (files, sockets, DNS, timers). V8 runs your JavaScript while libuv handles the asynchronous work behind the scenes.

Q17. What is callback hell and how do you avoid it?

Deeply nested callbacks that are hard to read and maintain. Avoid with promises, async/await, named functions, modularization and utilities like util.promisify.

Modules, Streams and Core APIs

Q18. What are streams and what types exist?

Streams process data in chunks instead of loading everything into memory. Types: Readable, Writable, Duplex (both) and Transform (modifies data while passing through). They are ideal for large files and network data.

```
fs.createReadStream('big.txt')
  .pipe(zlib.createGzip())
  .pipe(fs.createWriteStream('big.txt.gz'));
```

Q19. What is a Buffer?

A Buffer is a fixed-size chunk of raw binary memory outside the V8 heap, used to handle binary data such as files, TCP streams and images.

Q20. What is EventEmitter?

A core class (events module) that implements the observer pattern. Objects emit named events and listeners subscribe using on/once and are removed with off. Many Node APIs (streams, http servers) inherit from it.

```
const EventEmitter = require('events');
const e = new EventEmitter();
e.on('ping', msg => console.log(msg));
e.emit('ping', 'hello');
```

Q21. What is the difference between fork, spawn and exec in child_process?

spawn streams output from a command with no buffering limit. exec runs a command in a shell and buffers output (limited size). fork is a special spawn for Node scripts that creates an IPC channel for message passing.

Q22. What are worker threads and how do they differ from cluster?

worker_threads run JavaScript in parallel threads within one process and can share memory (SharedArrayBuffer), best for CPU-bound work. cluster forks multiple processes that share a server port, best for using all CPU cores for request handling.

Q23. What is the difference between fs.readFile and fs.createReadStream?

readFile loads the entire file into memory before giving you the data, which is fine for small files. createReadStream reads in chunks, using constant memory, which suits large files.

Express, Security and Production

Q24. What is middleware in Express?

Functions with access to req, res and next that run in sequence during a request cycle. They can modify the request/response, end the cycle, or pass control with next(). Error-handling middleware takes four arguments (err, req, res, next).

```
app.use((req, res, next) => {
  console.log(req.method, req.url);
  next();
});
```

Q25. How do you handle errors in Node.js applications?

Use try/catch with async/await, .catch on promises, and error-first callbacks. In Express, use a centralized error-handling middleware. Listen for unhandledRejection and uncaughtException to log and shut down gracefully, since the process may be in an unknown state.

Q26. How do you secure a Node.js application?

Validate and sanitize input, use helmet for HTTP headers, enable rate limiting and CORS properly, hash passwords with bcrypt or argon2, use HTTPS, keep dependencies updated (npm audit), store secrets in environment variables, and protect against injection, XSS and CSRF.

Q27. How do you manage environment variables and configuration?

Use process.env, with a .env file loaded by dotenv (or Node's --env-file flag) in development. Never commit secrets; use a secrets manager or the platform's environment configuration in production.

Q28. How do you scale a Node.js application?

Vertically with cluster or PM2 to use all cores; horizontally with multiple instances behind a load balancer (Nginx, cloud LB). Add caching (Redis), use message queues, keep services stateless and offload heavy work to workers.

Q29. What is PM2 and why use it?

PM2 is a production process manager for Node.js. It restarts crashed apps, runs them in cluster mode across CPU cores, manages logs, supports zero-downtime reloads and monitors resource usage.

Q30. What causes memory leaks in Node.js and how do you debug them?

Common causes: unbounded caches, global variables, forgotten timers or event listeners, and closures holding large objects. Debug using --inspect with Chrome DevTools heap snapshots, process.memoryUsage(), and tools like clinic.js.